Diagramas de Objetos II

Especificação de Classes, Propriedades e Associações

2006
©MJRL

Engenharia de Software

Nota

- O material apresentado nos slides seguintes foram, na sua maioria, extraídos de material disponibilizado por Eduardo Bezerra no site *sites.uol.com.br/eduardo.bezerra/papoo,* como suporte ao seu livro “Princípios de Análise e Projeto de Sistemas com UML” publicado pela editora Campus.

Introdução

- O **modelo de classes de especificação** é resultante de refinamentos no modelo de classes de domínio.
- Esse modelo é construído em paralelo com o modelo de interações.
- Após sua construção, o modelo de especificação é passado aos programadores para que eles o implementem.
- Aspectos a serem considerados:
  - Transformação de classes de domínio em classes de especificação
  - Criação de novas classes
  - Especificação de atributos
  - Especificação de operações
  - Especificação de associações

2006
©MJRL

Especificação de classes de fronteira

- Durante a análise, considera-se que há uma única classe de fronteira para cada ator.
- Na passagem para o modelo de especificação, algumas dessas classes podem resultar em várias outras.
  - Interface como seres humanos: ***projeto da interface gráfica*** produz o detalhamento das classes.
  - Equipamentos: uma ou mais classes para encapsular o protocolo de comunicação do equipamento.
    - » O mesmo vale para comunicação com outros sistemas.

Especificação de classes de fronteira

- É também usual a definição de um ***subsistema*** para representar a comunicação com outros sistemas.
- Reuso de classes para se obter alta produtividade e boa qualidade no produto de software em desenvolvimento.
  - Componentes podem ser adquiridos já prontos.
  - Recursos fornecidos pelo ambiente de programação sendo utilizado

2006
©MJRL

Especificação de classes de entidade

- Normalmente precisam de armazenamento persistente.
- As classes de entidades que devem ser armazenadas de modo persistente devem ser identificadas na especificação.
- Também devem ser identificados os padrões de acesso a cada classe de entidade cujos objetos devem ser persistentes.
  - A taxa de acesso a uma coleção influencia na política de armazenamento.

2006
©MJRL

Especificação de classes de controle

- Na especificação, deve-se analisar a verdadeira utilidade de cada classe de controle identificada durante a análise.
- Em casos de uso simples (e.g., manutenção de dados), classes de controle não são realmente necessárias.
  - classes de fronteira podem repassar os dados fornecidos pelos atores diretamente para as classes de entidade correspondentes.
- Em casos de uso complexos, uma classe de controle de análise pode resultar em duas ou mais classes de especificação.

2006
©MJRL

Especificação de atributos e operações

©MJRL

Sintaxe da UML para atributos

- A especificação completa de um atributo deve seguir a sintaxe:

[/]*visibilidade nome : tipo = valor-inicial*

  - Visibilidade e ***encapsulamento***.
  - Tipo pode ser um ***TAD***.
  - Tipo e valor inicial dependentes de linguagem.
  - Atributo ***derivado***
  - ***Escopo*** do atributo

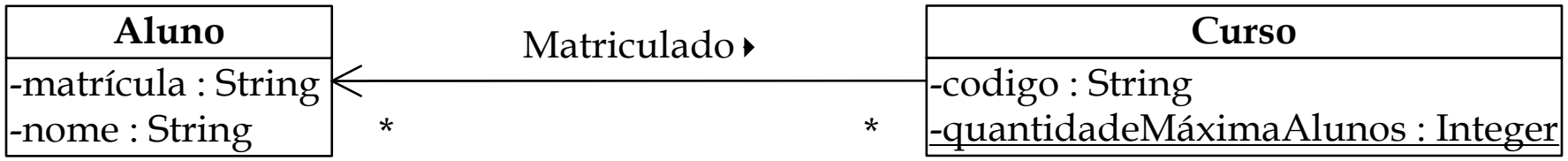
2006
©MJRL

Engenharia de Software

Exemplos (Sintaxe da UML para atributos)

- # *Método protegido*
- + *Método público*
- - *Método privado*

Cliente
#nome : String -dataNascimento : Data -telefone : String #/idade : int #limiteCrédito : Moeda = 500.0 -quantidadeClientes : int -idadeMédia : float



2006
©MJRL

Especificação de operações

- Operação: um processamento realizado por (um objeto de) uma classe.
  - Em termos de implementação: é uma rotina associada a uma classe.
- Na construção do modelo de interações, operações identificadas na análise são validadas e várias outras operações são identificadas.
  - Essas operações devem ser adicionadas ao diagrama de classes e documentadas através da definição de sua ***assinatura***.

2006
©MJRL

Sintaxe da UML para operações

- A especificação completa de um atributo deve seguir a sintaxe:

visibilidade nome(parâmetros): tipo-retorno {propriedades}

Onde, para cada parâmetro:

direção nome-parâmetro: tipo-parâmetro

- Visibilidade e ***encapsulamento***.
- Tipo pode ser um ***TAD***.
- Tipo de retorno e dos parâmetros dependentes de linguagem.
- ***Escopo*** da operação

2006
©MJRL

Engenharia de Software

Exemplos (Sintaxe da UML para operações)

Cliente
+obterNome() : String +definirNome(in umNome : String) +obterDataNascimento() : Data +definirDataNascimento(in umaData : Data) +obterTelefone() : String +definirTelefone(in umTelefone : String) +obterLimiteCrédito() : Moeda +definirLimiteCrédito(in umLimiteCrédito : float) +obterIdade() : int +obterQuantidadeClientes() : int +obterIdadeMédia() : float

2006
©MJRL

Engenharia de Software

Qualificadores para parâmetros

Direção	Significado
in	Parâmetro de entrada
out	Parâmetro de saída.
inout	Parâmetro de entrada e de saída.

Operações de criação e destruição

- Operações utilizadas freqüentemente nas interações entre objetos.
- Os parâmetros de uma operação de criação servem para iniciar atributos do objeto.
  - Regra prática: se o valor de um atributo é essencial para o significado do objeto, é mais adequado iniciar o seu valor logo na criação do objeto.
- Operações de destruição servem para liberar memória alocada dinamicamente.

2006
©MJRL

Operações de criação e destruição

- Operações de criação são identificadas na construção dos diagramas de interação.
- Operações de criação também são utilizadas para definir o estado inicial de um objeto que passa por um conjunto de **estados** *relevantes* durante a sua existência.

2006
©MJRL

Seletores e modificadores

- Princípio do *encapsulamento:* qualquer acesso a informações internas a uma classe deve ser realizado através de sua interface.
  - definida pelas operações de visibilidade pública.
- Para assegurar o encapsulamento da classe, todos os seus atributos com visibilidade privativa ou protegida.

2006
©MJRL

Seletores e modificadores

- Deve haver operações para obter acesso (***seletor***) e para modificar o valor (***modificador***) de cada atributo.
- Convenção utilizada:
  - o nome de um seletor é formado com o prefixo "*obter*" seguido do nome do atributo.
  - o nome de um modificador é construído com o prefixo "*definir*" seguido do nome do atributo.
- Nem sempre faz sentido ter um modificador e um seletor para um atributo...

2006
©MJRL

Especificação de associações

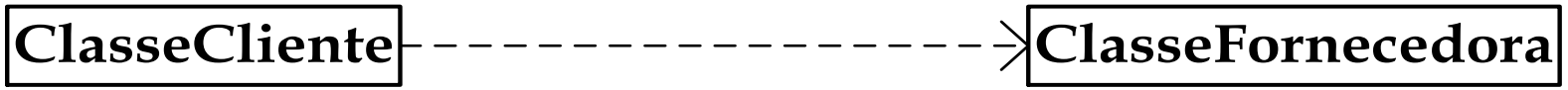
Relacionamentos de dependência

- Na UML, há três tipos de relacionamentos entre objetos:
  - Associações
  - Generalizações
  - Dependências
- No modelo de classes de domínio, os relacionamentos entre objetos foram identificados como associações.
- O relacionamento de dependência indica que uma classe depende dos serviços fornecidos por uma outra classe.
  - características de implementação.

2006
©MJRL

Relacionamentos de dependência

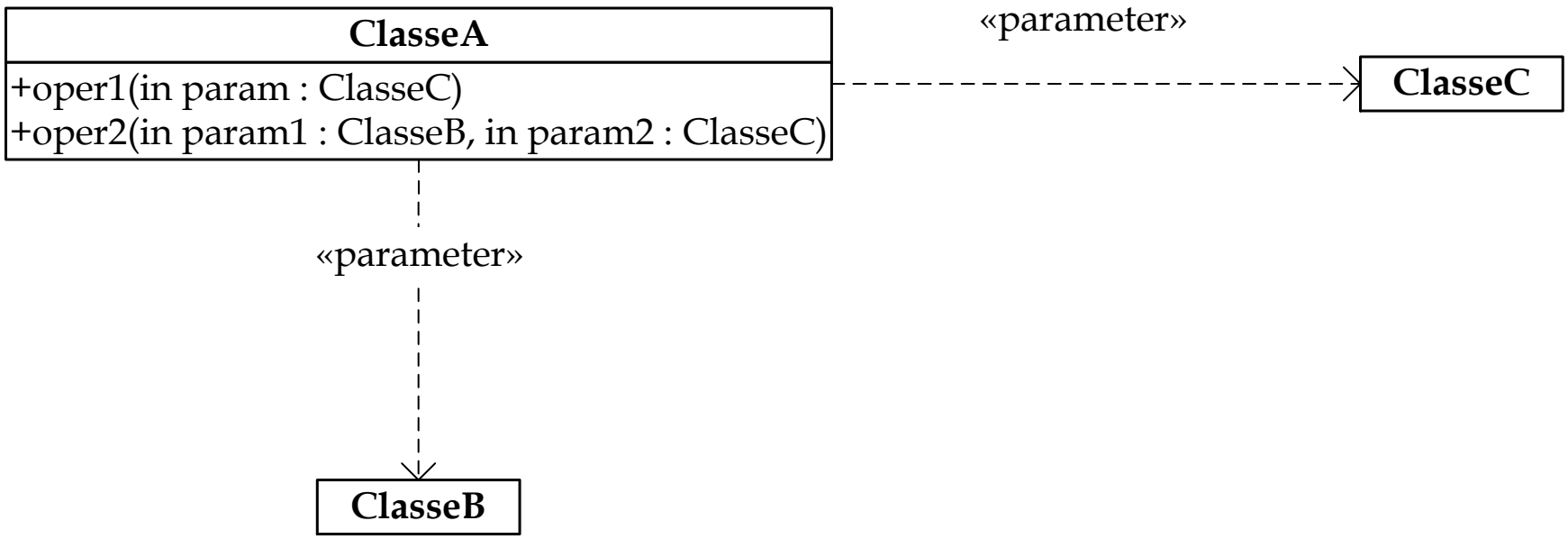
- Vários tipos de dependência entre duas classes podem existir.
  - *Dependência por atributo*.
  - *Dependência por variável global*.
  - *Dependência por variável local*.
  - *Dependência por parâmetro*.
- Representado através de uma linha direcionada e tracejada ligando as classes envolvidas.
- A direção é da classe dependente (***cliente***) para a classe da qual ela depende (***fornecedora***).
- Estereótipos: global, local, parameter.



2006
©MJRL

Engenharia de Software

Exemplo



Transformação de associações em dependências

- Na passagem para o modelo de especificação cada associação deve ser considerada para identificar se ela pode ser transformada em dependências.
- Razão: aumentar o encapsulamento das classes
  - Dependência por atributo torna as classes envolvidas mais dependentes uma da outra.
  - Quanto menos dependências estruturais houver no modelo de classes, maior é o encapsulamento das classes constituintes.

2006
©MJRL

Transformação de associações em dependências

- Associações entre classes de fronteira e controladores:
  - classe de fronteira para esse ator: essa classe pode manter um relacionamento estrutural com o controlador (*composição*).
  - classes de fronteira para equipamentos ou outros sistemas: *interfaces,* dependência não estrutural.

2006
©MJRL

Transformação de associações em agregações ou composições

- Uso de agregação na modelagem de classes de domínio.
- Alternativa: utilização de agregação somente durante o modelo de especificação.
- Em qualquer um dos casos, o que deve ser considerado é que a agregação possui uma forma ainda mais forte, a **composição**.
- A definição de composições é mais adequada para o modelo de especificação.

2006
©MJRL

Transformação de associações em agregações ou composições

- Na agregação:
  - Os objetos que fazem parte do todo são criados e destruídos independentemente deste último.
  - Um objeto parte pode ser utilizado para compor diversos objetos todos.
  - A destruição de um desses objetos todo não implica na destruição do objeto parte.

2006
©MJRL

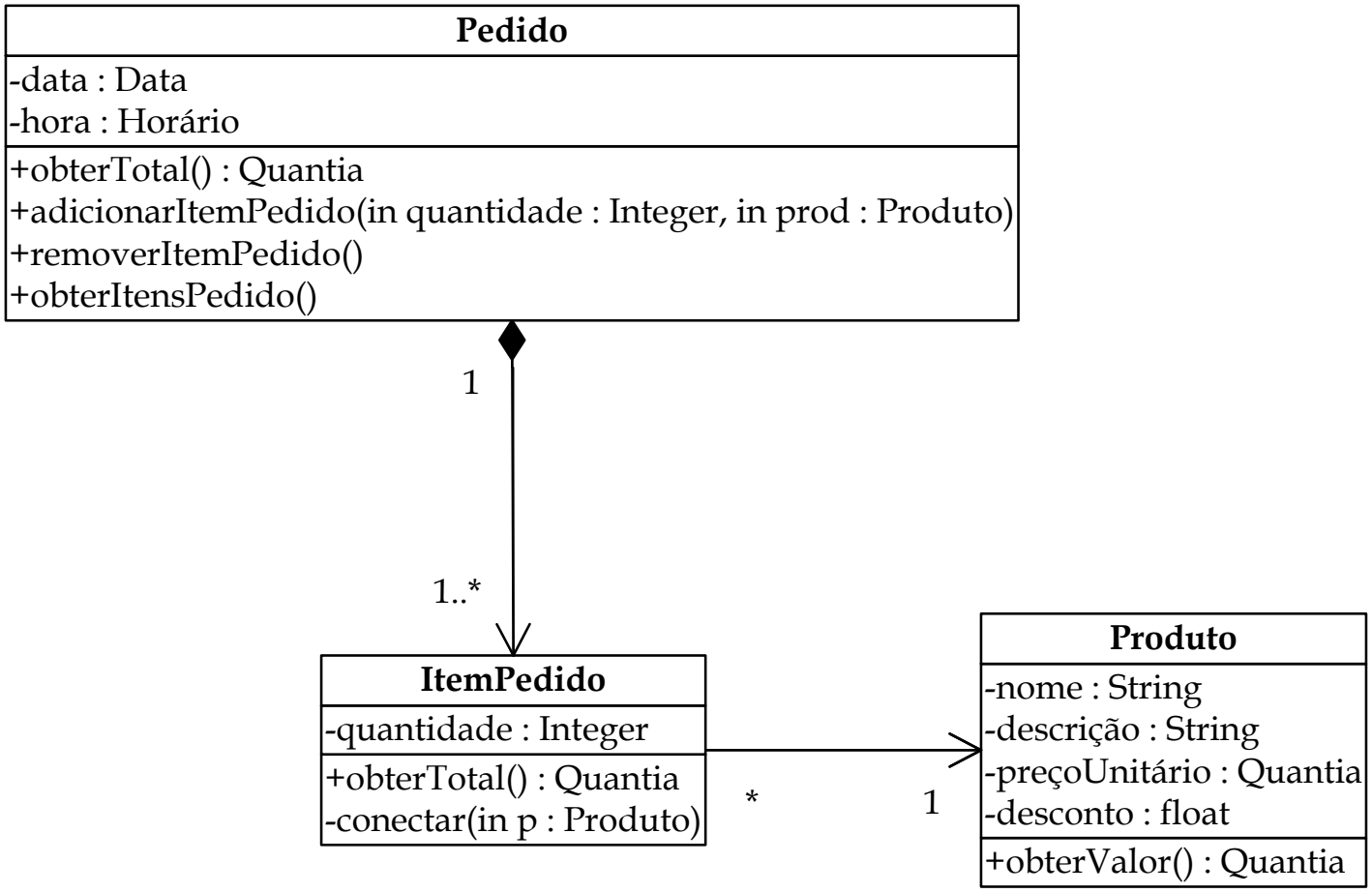
Transformação de associações em agregações ou composições

- Na composição:
  - os objetos *parte* pertencem a um único *todo*.
  - objetos parte são sempre criados e destruídos pelo objeto todo.
  - Usualmente os clientes do objeto todo não têm conhecimento da estrutura das partes.
    - » No objeto composto, são definidas operações para adicionar e remover objetos componentes.

2006
©MJRL

Engenharia de Software

Transformação de associações em agregações ou composições

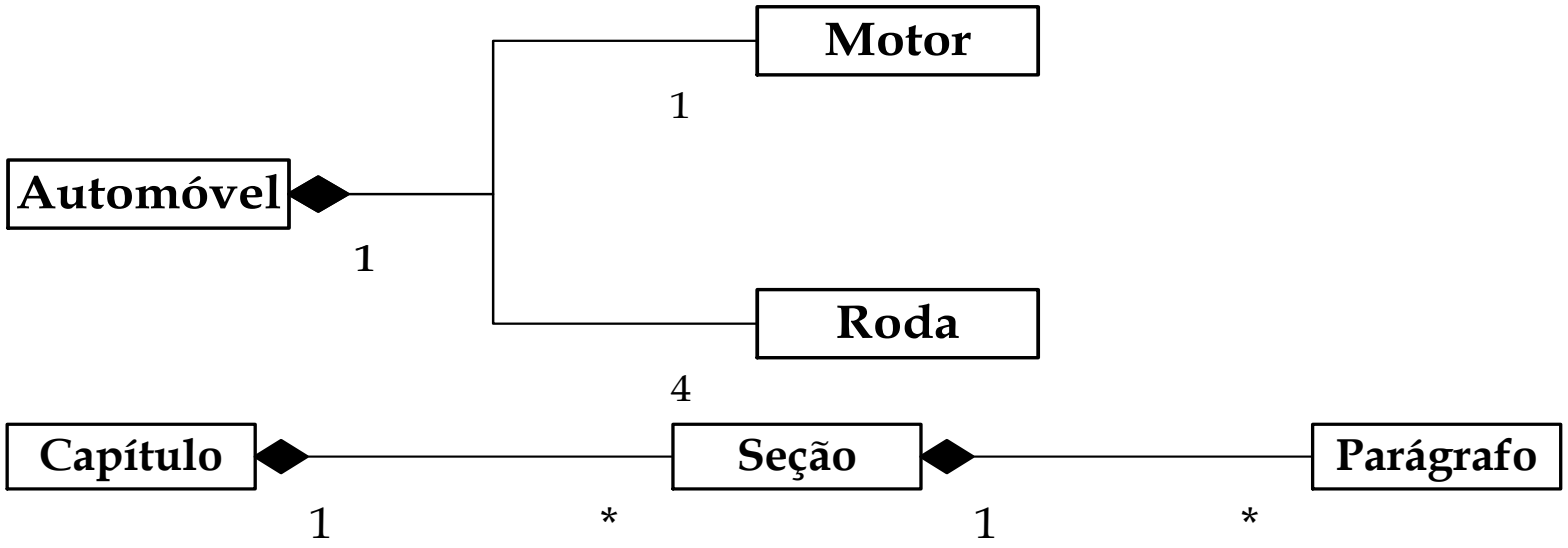


2006
©MJRL

Engenharia de Software

Transformação de associações em agregações ou composições

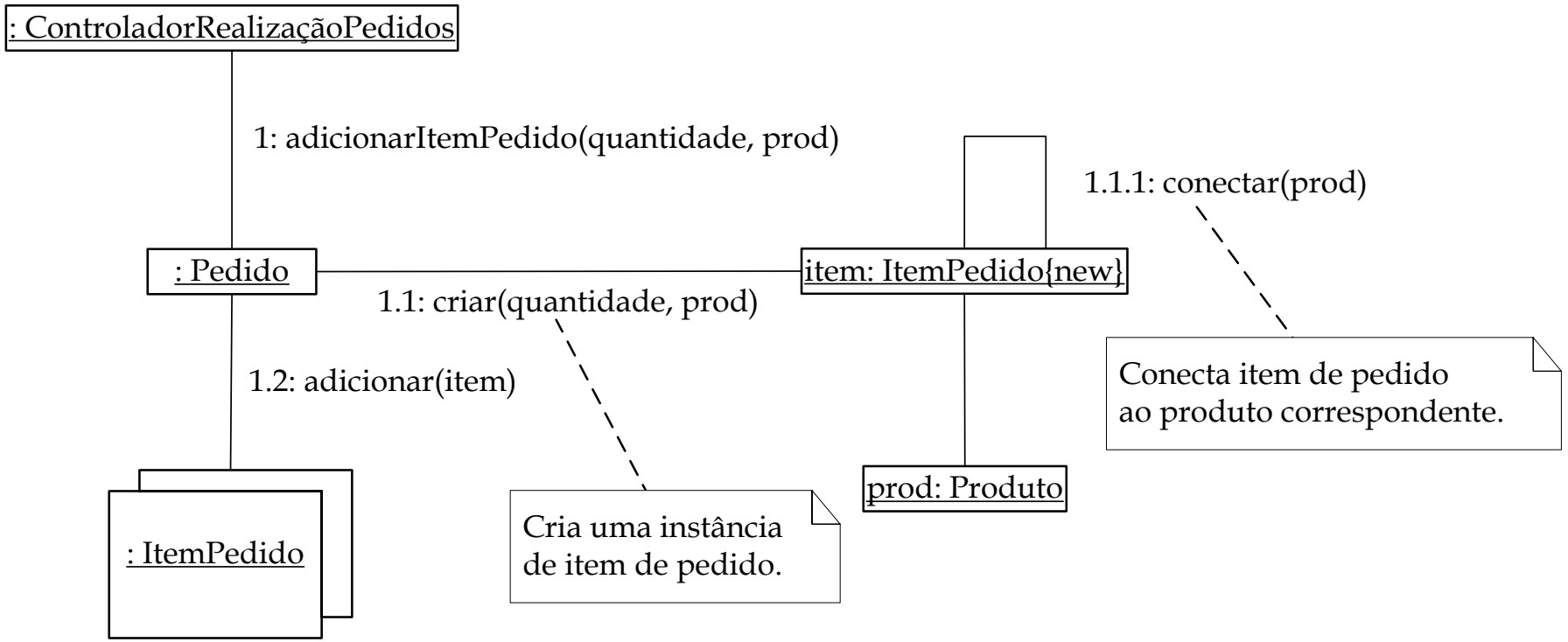
- Note que composições (agregações) podem se estender por diversos níveis, formando **hierarquias de composição** (**agregação**).



2006
©MJRL

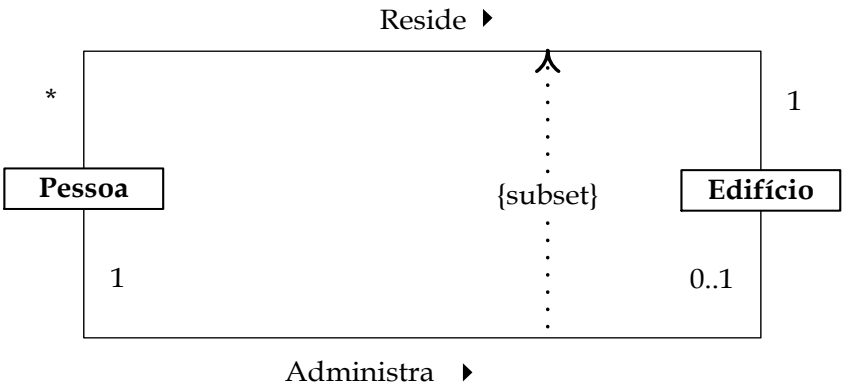
Transformação de associações em composições

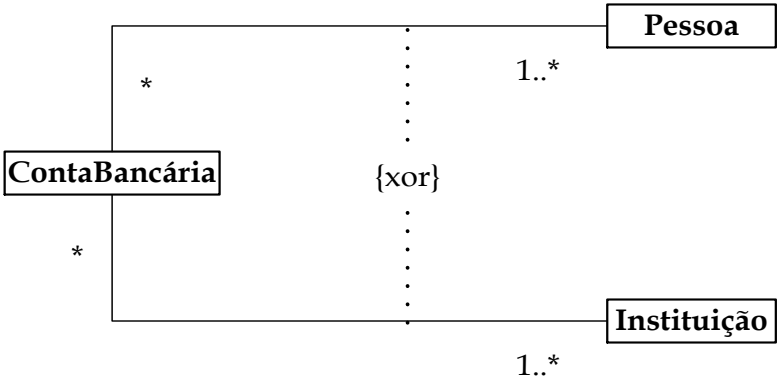
- Em geral, em uma agregação (composição), o todo tem prioridade para criar suas partes.



2006
©MJRL

Restrições sobre associações





- Restrições podem ser adicionadas sobre uma associação para condicionar a forma pela qual a associação deve ser implementada.
- Duas das restrições sobre associações predefinidas pela UML são:
  - **subset** (subconjunto)
  - **xor** (ou exclusivo).

Navegabilidade de associações

- Associações, agregações e composições podem ser **bidirecionais** e **unidirecionais**.
- Uma associação bidirecional indica que há um conhecimento mútuo entre os objetos associados.
- Na UML, associações são, por omissão, navegáveis em ambos os sentidos.

2006
©MJRL

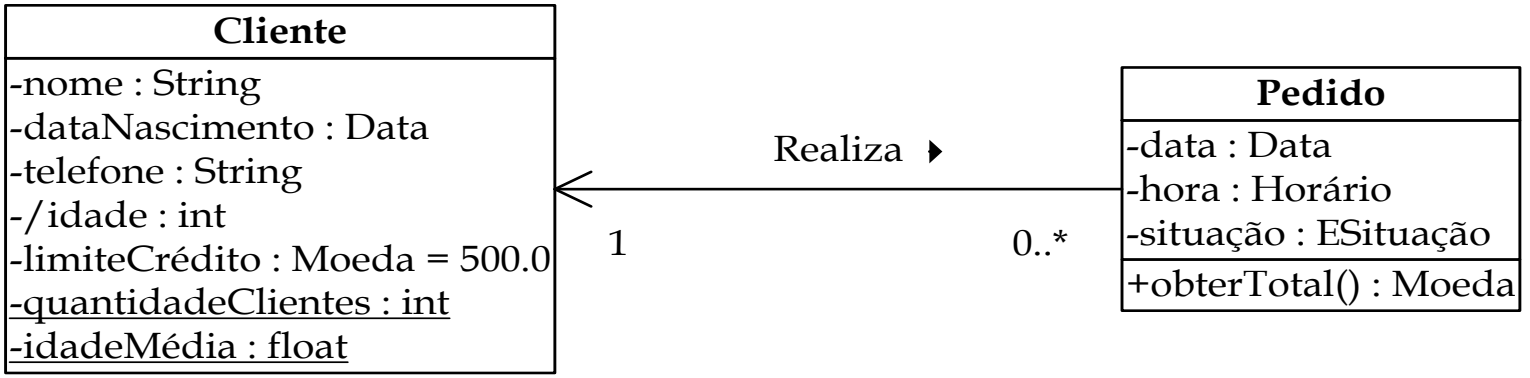
Navegabilidade de associações

- Uma associação unidirecional é representada adicionando-se um sentido à seta da associação.
- No modelo de classes de especificação, a navegabilidade de todas as associações deve ser definida.
  - Algumas associações permanecem bidirecionais.
  - Se não houver essa necessidade, recomenda-se transformar a associação em unidirecional.

2006
©MJRL

Navegabilidade de associações

- A escolha do sentido da navegabilidade pode ser feita através dos diagramas de interação.
  - Mensagens influenciam na existência ou não de navegabilidade em um sentido.



Definindo a implementação de associações

- Para associações de conectividade um para um, a implementação é trivial: é definido um atributo do tipo Cb na classe Ca.
- Navegabilidade bidirecional: aplica-se o procedimento acima para as duas classes.
- Portanto, em associações um para um, não há necessidade de um refinamento adicional do diagrama de classes.

2006
©MJRL

Definindo a implementação de associações

- Em associações de conectividade *um para muitos* ou *muitos para muitos*, detalhes adicionais podem ser representados para esclarecer como implementar tais associações.
- O detalhamento de associações se baseia em classes que representam coleções de elementos.

2006
©MJRL

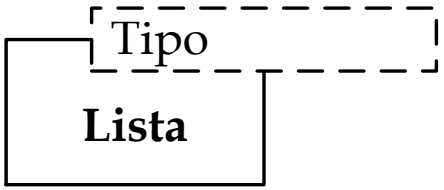
Classe Parametrizada

- Uma coleção pode ser representada em um DC através uma **classe parametrizada** (template).
  - É uma classe é utilizada para definir outras classes.
  - Possui operações ou atributos cuja definição é feita em função de um ou mais parâmetros.
- Uma coleção pode ser definida a partir de uma classe parametrizada, onde o parâmetro é o tipo do elemento da coleção.

2006
©MJRL

Classe Parametrizada

- Há duas notações possíveis na UML para uma classe parametrizada.
- Exemplo: classe parametrizada **Lista**:



Lista<Tipo>

Refinando associações de conectividade um para muitos

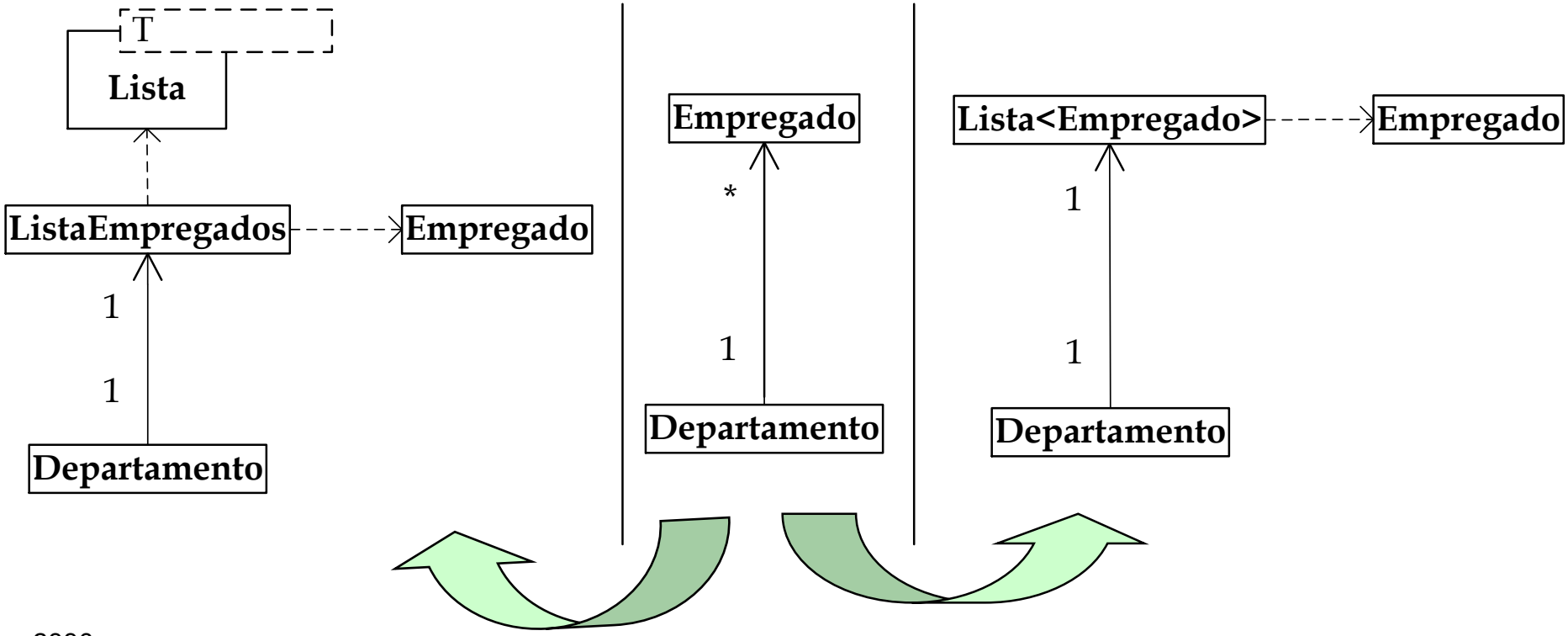
- Classes parametrizadas podem ser utilizadas para detalhar associações *um para muitos*.
  - Idéia básica: definir uma classe parametrizada cujo parâmetro é a classe correspondente ao lado *muitos* da associação.
  - O caso muitos para muitos é bastante semelhante ao refinamento das associações um para muitos.
- A utilização da classe parametrizada resulta na transformação de uma associação *um para muitos* em uma associação *um para um*.

2006
©MJRL

Engenharia de Software

Refinando associações de conectividade um para muitos

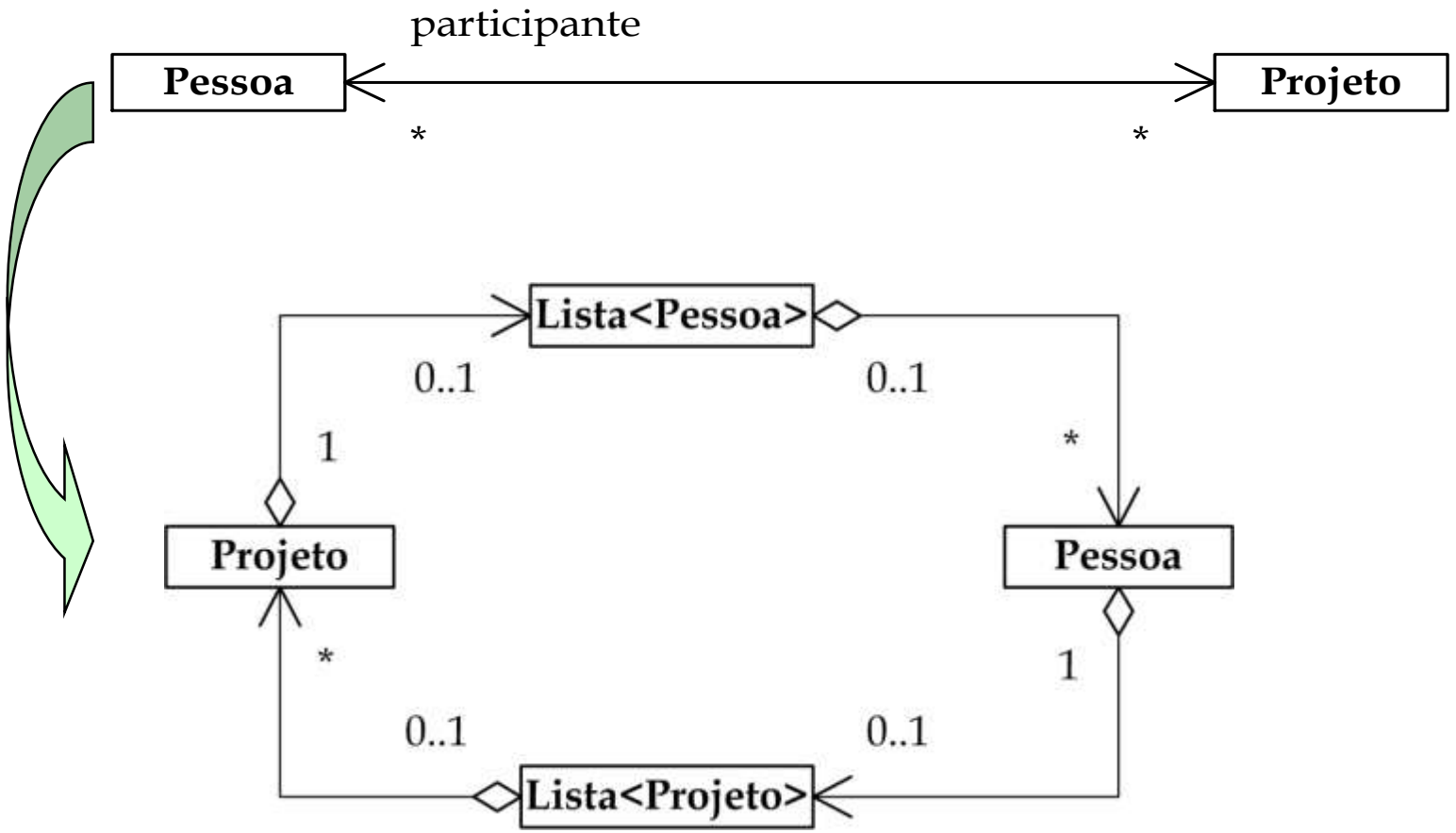
- Formas alternativas para representar uma associação um para muitos.



2006
©MJRL

Engenharia de Software

Refinando associações de conectividade muitos para muitos



2006
©MJRL

Refinando classes associativas, associações ternárias

- A solução mais usual é criar uma classe de especificação para substituir a classe associativa.
  - Uma vez feito isso, o problema se transforma no refinamento de associações um para muitos ou muitos para muitos.
- O mesmo procedimento pode ser aplicado em associações ternárias.

2006
©MJRL

Engenharia de Software

Refinando classes associativas, associações ternárias

